Clasificación de Escenas Naturales mediante Redes Neuronales Convolucionales

Charbel Isaias Chávez Zavala
A01711950
Inteligencia Artificial Avanzada para la Ciencia de Datos II

6 de noviembre de 2026

Resumen

Este trabajo presenta el desarrollo de un modelo de aprendizaje profundo para clasificar fotografías de escenas naturales en seis categorías: edificios, bosque, glaciar, montaña, mar y calle. Se utilizó el conjunto de datos *Intel Image Classification*, compuesto por más de 24,000 imágenes RGB de 150×150 píxeles.

Se implementó una red neuronal convolucional (CNN) construida desde cero con TensorFlow/Keras, sin pesos preentrenados ni técnicas de regularización, con el fin de establecer un modelo base contra el cual medir mejoras posteriores. El modelo alcanzó una exactitud de 83.33 % y un F1-score macro de 0.8355 sobre el conjunto de prueba. El análisis de las curvas de entrenamiento revela un sobreajuste marcado a partir de la sexta época, y la matriz de confusión muestra que la mitad de los errores se concentra en dos pares de clases visualmente similares: glaciar–montaña y edificios–calle.

1. Introducción

El aprendizaje profundo (*deep learning*) es una rama del aprendizaje automático basada en redes neuronales artificiales con múltiples capas, capaces de aprender representaciones jerárquicas de los datos directamente a partir de la información cruda. A diferencia de los modelos de aprendizaje automático clásicos, que dependen de variables diseñadas manualmente por un experto, una red profunda aprende por sí misma qué características son relevantes para resolver la tarea.

En el ámbito de la visión por computadora, las **redes neuronales convolucionales** (CNN, por sus siglas en inglés) se han consolidado como la arquitectura de referencia desde que AlexNet ganó la competencia ImageNet en 2012. Su diseño se inspira en la organización de la corteza visual: en lugar de conectar cada píxel con cada neurona, las capas convolucionales aplican pequeños filtros que recorren la imagen y detectan patrones locales. Las primeras capas aprenden patrones simples (bordes, colores), y las capas más profundas los combinan en patrones cada vez más complejos (texturas, formas, objetos).

El problema abordado en este trabajo es de **clasificación multiclase** supervisada: dada una fotografía, el modelo debe asignarle una de seis categorías de escena.

1.1. Objetivo

El objetivo de este proyecto es desarrollar una red neuronal convolucional, implementada con TensorFlow, capaz de clasificar fotografías de escenas naturales en seis categorías. En esta primera etapa se construye un **modelo base** sin regularización, cuyo desempeño y limitaciones servirán como referencia para las mejoras de la siguiente etapa.

1.2. Descripción del conjunto de datos

El conjunto de datos utilizado es *Intel Image Classification*, publicado originalmente por Intel para un reto de clasificación de imágenes y disponible en Kaggle. Contiene fotografías RGB de 150×150 píxeles distribuidas en tres carpetas:

- `seg_train`: 14,034 imágenes etiquetadas, destinadas al entrenamiento.
- `seg_test`: 3,000 imágenes etiquetadas, destinadas a la evaluación.
- `seg_pred`: 7,301 imágenes sin etiqueta, utilizadas para realizar predicciones sobre datos nuevos.

Cuadro 1: Descripción de las clases del conjunto de datos.

Clase	**Descripción**
`buildings`	Fachadas de edificios y construcciones urbanas.
`forest`	Bosques y zonas con vegetación densa.
`glacier`	Glaciares, hielo y paisajes nevados.
`mountain`	Montañas y paisajes montañosos.
`sea`	Mar, costas y cuerpos de agua.
`street`	Calles y vías urbanas.

2. Análisis Exploratorio de Datos

El análisis exploratorio se enfocó en verificar el balance de las clases y en inspeccionar visualmente las imágenes para anticipar qué clases podrían resultar difíciles de distinguir.

Cuadro 2: Número de imágenes por clase en los conjuntos de entrenamiento y prueba.

Clase	**Train**	**% Train**	**Test**	**% Test**
`buildings`	2,191	15.61	437	14.57
`forest`	2,271	16.18	474	15.80
`glacier`	2,404	17.13	553	18.43
`mountain`	2,512	17.90	525	17.50
`sea`	2,274	16.20	510	17.00
`street`	2,382	16.97	501	16.70
Total	14,034	100	3,000	100

Las clases se encuentran **razonablemente balanceadas**: la clase más pequeña (`buildings`, 15.61 %) y la más grande (`mountain`, 17.90 %) difieren en poco más de dos puntos porcentuales, y las proporciones se mantienen similares entre entrenamiento y prueba. Un clasificador trivial que siempre predijera la clase mayoritaria obtendría apenas 17.50 % de exactitud en prueba. Por ello, a diferencia de un problema desbalanceado, la exactitud (*accuracy*) es una métrica adecuada; aun así, se reportan también precision, recall y F1-score por clase para identificar las categorías problemáticas.

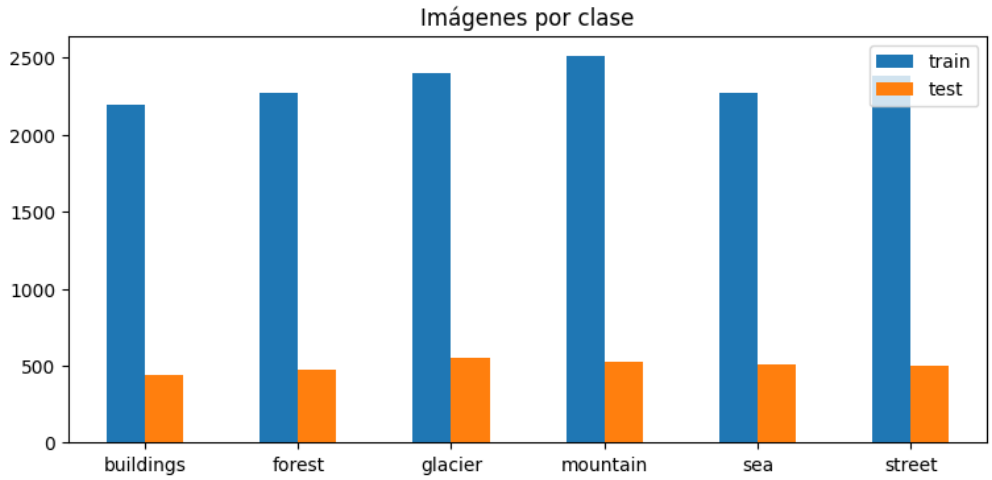


Figura 1: Distribución de imágenes por clase en los conjuntos de entrenamiento y prueba.

La Figura 2 muestra ejemplos aleatorios de cada clase. La inspección visual anticipa dos fuentes de confusión: las clases `glacier` y `mountain` comparten picos nevados, rocas y cielos amplios, mientras que `buildings` y `street` comparten fachadas, ventanas y elementos urbanos. En muchas fotografías ambos elementos aparecen simultáneamente (por ejemplo, una calle flanqueada por edificios), por lo que la etiqueta depende de qué elemento domina la escena.

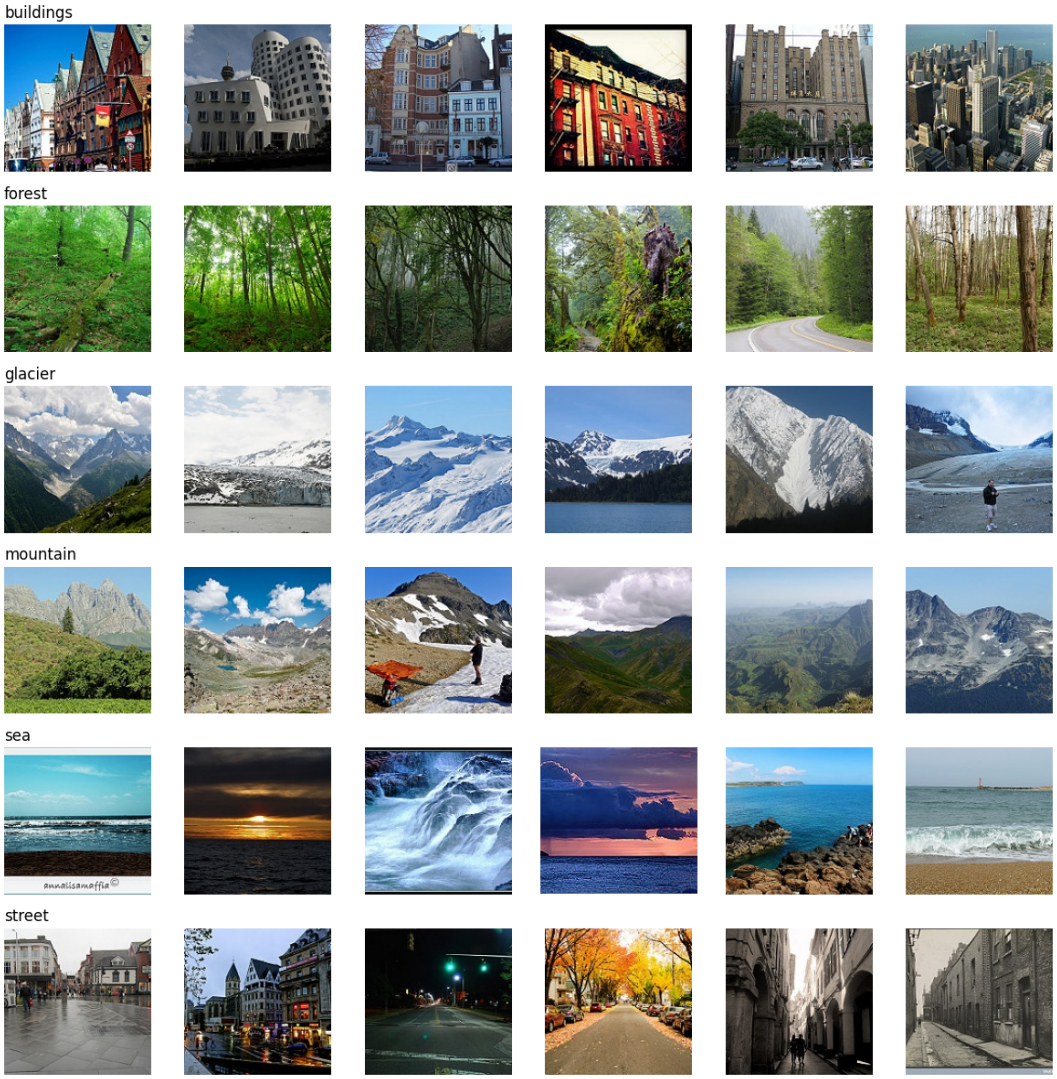


Figura 2: Ejemplos aleatorios del conjunto de entrenamiento (una fila por clase).

3. Preprocesamiento de Datos

A diferencia de los datos tabulares, las imágenes no requieren imputación ni codificación de variables categóricas: la propia red aprende las características a partir de los píxeles. El preprocesamiento se limitó a las transformaciones descritas a continuación, implementadas con `tf.data` y la función `keras.utils.image_dataset_from_directory`, que infiere la etiqueta de cada imagen a partir del nombre de su carpeta.

3.1. Redimensionamiento

Todas las imágenes se cargan con un tamaño fijo de $150 \times 150 \times 3$, que coincide con la resolución nativa del conjunto de datos. Las pocas imágenes con dimensiones distintas se redimensionan automáticamente al cargarse, garantizando que todas las entradas de la red

tengan la misma forma.

3.2. Normalización

Los valores de los píxeles, originalmente enteros en el rango $[0, 255]$, se escalan al intervalo $[0, 1]$:

$$x' = \frac{x}{255} \tag{1}$$

Esta normalización se implementó como la primera capa del modelo (`layers.Rescaling(1/255)`) en lugar de aplicarla por separado a los datos. De este modo, el modelo guardado recibe imágenes crudas y aplica la misma transformación al hacer predicciones sobre datos nuevos, eliminando el riesgo de olvidar el preprocesamiento en producción. Trabajar con entradas en una escala pequeña favorece una convergencia más estable del descenso de gradiente.

3.3. División de los datos

Se utilizaron tres subconjuntos con roles distintos:

- **Entrenamiento** (80 % de `seg_train`, 11,228 imágenes): para ajustar los pesos de la red.
- **Validación** (20 % de `seg_train`, 2,806 imágenes): para monitorear el sobreajuste durante el entrenamiento y seleccionar el mejor modelo.
- **Prueba** (`seg_test` completo, 3,000 imágenes): reservado exclusivamente para la evaluación final.

La partición entre entrenamiento y validación se realizó con semilla fija (`seed = 42`) para garantizar la reproducibilidad y asegurar que todos los experimentos futuros se comparen sobre exactamente las mismas imágenes de validación. El conjunto de prueba proviene de una carpeta independiente, por lo que no existe fuga de información (*data leakage*) hacia el entrenamiento.

3.4. Eficiencia del pipeline

Los conjuntos se almacenan en memoria tras la primera época (`cache()`) y los lotes se preparan en paralelo mientras la GPU entrena el lote anterior (`prefetch()`). Con esto, el tiempo por época se redujo de 59 segundos en la primera época a aproximadamente 11 segundos en las siguientes.

4. Red Neuronal Convolucional (Modelo Base)

4.1. Componentes de la arquitectura

Capa convolucional. Aplica K filtros de tamaño 3×3 que recorren la imagen. Cada filtro calcula, en cada posición, una suma ponderada de los píxeles de su vecindad más un sesgo,

produciendo un mapa de activación que indica dónde aparece el patrón que el filtro aprendió a detectar:

$$y_{i,j,k} = b_k + \sum_{u=0}^{2} \sum_{v=0}^{2} \sum_{c} w_{u,v,c,k}\; x_{i+u,\, j+v,\, c} \tag{2}$$

A diferencia de una capa densa, los pesos del filtro se **comparten** en todas las posiciones de la imagen, lo que reduce drásticamente el número de parámetros y permite detectar un mismo patrón sin importar dónde aparezca.

Función de activación ReLU. Introduce no linealidad, permitiendo a la red aprender relaciones complejas:

$$\mathrm{ReLU}(z) = \mathrm{máx}(0, z) \tag{3}$$

Max pooling. Reduce a la mitad el alto y el ancho de cada mapa de activación conservando el valor máximo de cada ventana de 2×2. Esto disminuye el costo computacional, aporta cierta invariancia a pequeños desplazamientos y permite que las capas siguientes "vean" regiones más amplias de la imagen original.

Softmax. La última capa produce un vector de seis valores que se transforma en una distribución de probabilidad sobre las clases:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{6} e^{z_j}} \tag{4}$$

La clase predicha es la de mayor probabilidad.

4.2. Arquitectura del modelo

El modelo sigue el patrón clásico de las arquitecturas tipo VGG: bloques de convolución 3×3 seguidos de *max pooling*, en los que el número de filtros aumenta conforme la resolución espacial disminuye. Las capas convolucionales funcionan como **extractor de características**, y las capas densas finales como **clasificador**.

Cuadro 3: Arquitectura de la CNN base.

Capa	**Configuración**	**Salida**	**Parámetros**
Entrada + Rescaling	$\div 255$	$150 \times 150 \times 3$	0
Conv2D + ReLU	32 filtros 3×3	$150 \times 150 \times 32$	896
MaxPooling2D	2×2	$75 \times 75 \times 32$	0
Conv2D + ReLU	64 filtros 3×3	$75 \times 75 \times 64$	18,496
MaxPooling2D	2×2	$37 \times 37 \times 64$	0
Conv2D + ReLU	128 filtros 3×3	$37 \times 37 \times 128$	73,856
MaxPooling2D	2×2	$18 \times 18 \times 128$	0
Conv2D + ReLU	128 filtros 3×3	$18 \times 18 \times 128$	147,584
MaxPooling2D	2×2	$9 \times 9 \times 128$	0
Flatten	–	10,368	0
Dense + ReLU	128 neuronas	128	1,327,232
Dense + Softmax	6 neuronas	6	774
Total			**1,568,838**

Un aspecto relevante de esta arquitectura es la distribución de los parámetros: la capa densa posterior a `Flatten` concentra 1,327,232 parámetros, es decir, el **84.6 %** del total, mientras que las cuatro capas convolucionales suman apenas 240,832. Esta concentración de capacidad en una sola capa la convierte en la principal candidata a memorizar el conjunto de entrenamiento.

El modelo se diseñó sin técnicas de regularización, con el fin de obtener una línea base clara y poder medir después el efecto aislado de cada técnica.

4.3. Función de pérdida

Para clasificación multiclase se utilizó la entropía cruzada categórica (*sparse categorical cross-entropy*), que penaliza con mayor intensidad las predicciones incorrectas realizadas con alta confianza:

$$L = -\frac{1}{n} \sum_{i=1}^{n} \log \left(\hat{y}_{i,\, y_i} \right) \tag{5}$$

donde $\hat{y}_{i,\, y_i}$ es la probabilidad que el modelo asigna a la clase verdadera de la imagen i. Se utiliza la variante *sparse* porque las etiquetas se representan como enteros (0–5) en lugar de vectores *one-hot*.

4.4. Optimización y entrenamiento

Los pesos se ajustaron con el optimizador **Adam**, una variante del descenso de gradiente que adapta la tasa de aprendizaje de cada parámetro a partir de estimaciones del primer y segundo momento de sus gradientes. Los hiperparámetros utilizados fueron:

- Tasa de aprendizaje: $\alpha = 10^{-3}$.

- Tamaño de lote (*batch size*): 32 imágenes (351 lotes por época).
- Épocas: 20.
- Semilla global: 42.

Durante el entrenamiento se utilizó un *callback* `ModelCheckpoint` que guarda los pesos de la época con mayor exactitud de validación. Así, el modelo evaluado corresponde a la mejor época y no necesariamente a la última. El entrenamiento se realizó en una GPU de Google Colab.

5. Resultados

5.1. Curvas de entrenamiento

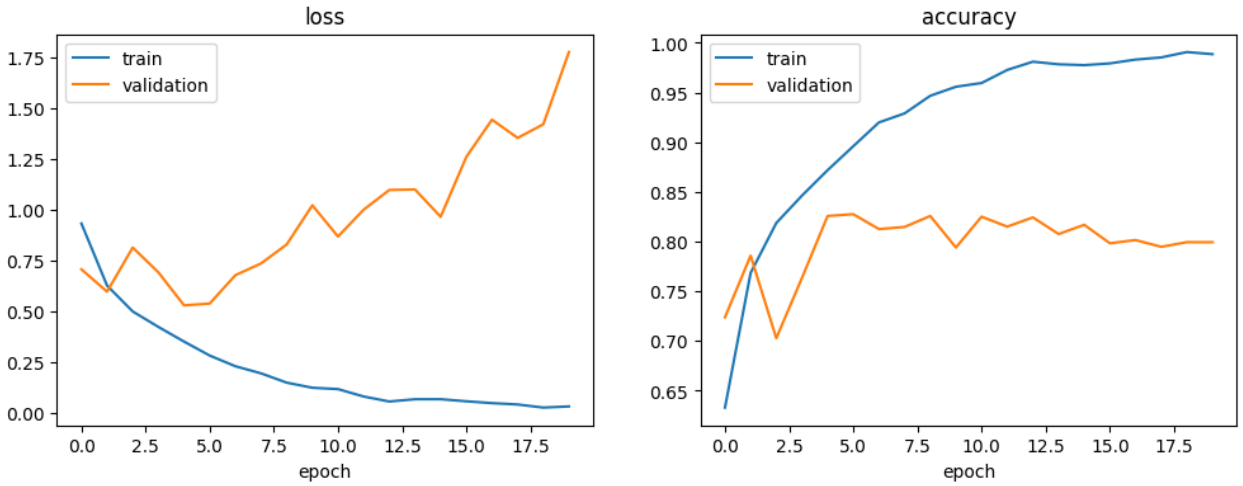


Figura 3: Evolución de la pérdida (izquierda) y de la exactitud (derecha) en entrenamiento y validación a lo largo de las 20 épocas.

Cuadro 4: Métricas en épocas representativas del entrenamiento.

Época	Loss train	Acc. train	Loss val	Acc. val
1	0.9349	0.6330	0.7092	0.7238
5	0.3534	0.8718	**0.5318**	0.8257
6	0.2847	0.8959	0.5406	**0.8275**
10	0.1262	0.9557	1.0244	0.7940
15	0.0698	0.9775	0.9677	0.8168
20	0.0343	0.9885	1.7787	0.7994

Las curvas muestran un patrón claro de **sobreajuste**. Durante las primeras cinco épocas ambas pérdidas disminuyen juntas y el modelo aprende patrones generalizables. A partir de

la sexta época las curvas se separan: la pérdida de entrenamiento sigue bajando hasta 0.0343, mientras que la de validación alcanza su mínimo en la época 5 (0.5318) y después crece hasta 1.7787 en la época 20, más de tres veces su valor mínimo. De forma análoga, la exactitud de entrenamiento llega a 98.85 %, mientras que la de validación se estanca alrededor de 80–83 %, dejando una brecha final de casi 19 puntos porcentuales.

El hecho de que la pérdida de validación aumente mientras la exactitud de validación se mantiene relativamente estable indica que el modelo se vuelve **sobreconfiado**: sigue acertando en una proporción similar de imágenes, pero sus errores se cometen con probabilidades cada vez más cercanas a 1, lo cual la entropía cruzada penaliza fuertemente.

El `ModelCheckpoint` conservó el modelo de la época 6 (exactitud de validación de 82.75 %). Sin este mecanismo, el modelo final de la época 20 habría obtenido solo 79.94 % en validación. Además, 14 de las 20 épocas resultaron innecesarias, lo que indica que el entrenamiento podría detenerse mucho antes.

5.2. Métricas de evaluación

Cuadro 5: Métricas globales del modelo base (pesos de la época 6).

Conjunto	**Accuracy**	**Precision (macro)**	**Recall (macro)**	**F1-Score (macro)**
Entrenamiento*	0.8959	–	–	–
Validación	0.8275	0.8317	0.8291	0.8294
Prueba	0.8333	0.8380	0.8366	0.8355

*Exactitud de entrenamiento reportada por Keras durante la época 6.

El desempeño en validación (82.75 %) y en prueba (83.33 %) difiere en menos de un punto porcentual, lo que confirma que el conjunto de validación es un buen estimador del desempeño sobre datos no vistos y que puede utilizarse con confianza para comparar las versiones mejoradas del modelo. La exactitud de 83.33 % es muy superior al 17.50 % de un clasificador trivial, por lo que el modelo sí aprendió patrones visuales útiles.

Cuadro 6: Métricas por clase sobre el conjunto de prueba.

Clase	**Precision**	**Recall**	**F1-Score**	**Soporte**
`buildings`	0.8115	0.8375	0.8243	437
`forest`	0.9354	0.9768	**0.9556**	474
`glacier`	0.8186	0.7179	**0.7649**	553
`mountain`	0.7252	0.8495	0.7825	525
`sea`	0.8482	0.8216	0.8347	510
`street`	0.8891	0.8164	0.8512	501
Macro	0.8380	0.8366	0.8355	3,000

La clase `forest` es prácticamente resuelta por el modelo (F1 = 0.9556), lo cual es consistente con que su color verde y su textura de follaje son muy distintivos respecto al resto. En el extremo opuesto, `glacier` es la clase más difícil (F1 = 0.7649): su recall de 71.79 % indica que el modelo deja de reconocer a más de una de cada cuatro imágenes de glaciar. A su vez, `mountain` tiene la precision más baja (72.52 %), es decir, muchas imágenes que el modelo etiqueta como montaña pertenecen en realidad a otra clase.

5.3. Matriz de confusión

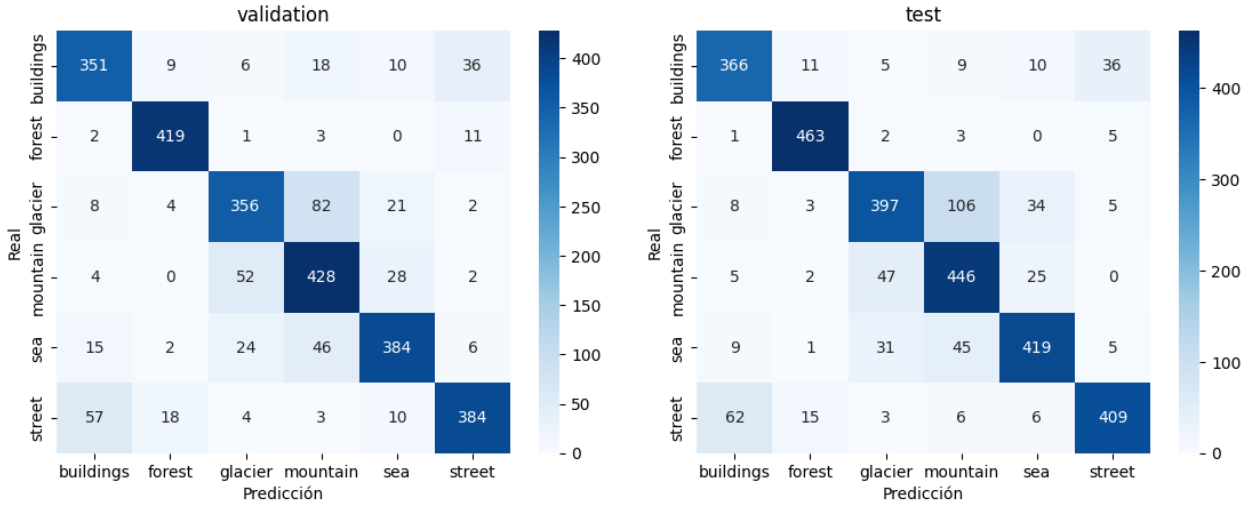


Figura 4: Matrices de confusión sobre los conjuntos de validación (izquierda) y prueba (derecha). Las filas corresponden a la clase real y las columnas a la predicha.

Cuadro 7: Confusiones más frecuentes en el conjunto de prueba.

Clase real	Predicción	Errores	% de la clase real
`glacier`	`mountain`	106	19.2
`street`	`buildings`	62	12.4
`mountain`	`glacier`	47	9.0
`sea`	`mountain`	45	8.8
`buildings`	`street`	36	8.2
`glacier`	`sea`	34	6.1

De las 3,000 imágenes de prueba, el modelo clasificó correctamente 2,500 y cometió 500 errores. Estos no se distribuyen de manera uniforme:

- El par `glacier`–`mountain` acumula 153 errores (en ambas direcciones), el **30.6 %** de todos los errores del modelo. Por sí sola, la confusión de glaciar como montaña representa el 19.2 % de todas las imágenes de glaciar.

- El par `buildings`–`street` acumula 98 errores, otro **19.6 %**.
- Las imágenes de `sea` se confunden principalmente con `mountain` (45) y `glacier` (31), probablemente por costas con montañas al fondo y por la presencia de agua y hielo en los paisajes de glaciar.

En conjunto, los dos pares de clases anticipados durante el análisis exploratorio explican la mitad de los errores. Estas confusiones reflejan en parte una ambigüedad real de las etiquetas, ya que una misma fotografía puede contener legítimamente un glaciar y una montaña, o una calle y varios edificios. Los patrones de confusión son muy similares en validación y en prueba, lo que confirma que se trata de debilidades sistemáticas del modelo y no de variaciones aleatorias.

5.4. Predicciones sobre datos nuevos

Para probar el modelo con imágenes nunca vistas y sin etiqueta, se realizaron predicciones sobre 12 imágenes seleccionadas aleatoriamente de `seg_pred`. Para cada una se muestra la clase con mayor probabilidad y su nivel de confianza.

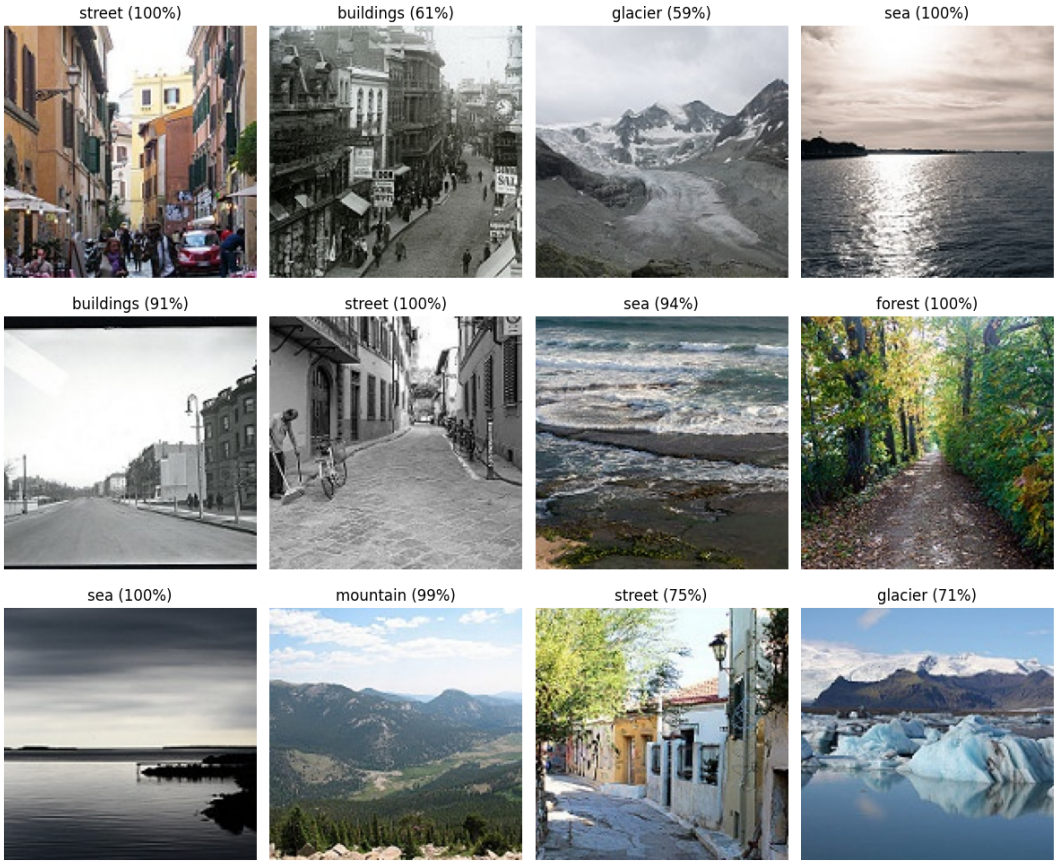


Figura 5: Predicciones del modelo base sobre 12 imágenes del conjunto `seg_pred`, con la clase predicha y su probabilidad.

Las predicciones visualmente correctas coinciden con escenas típicas de cada clase (bosques, mar abierto, calles con transeúntes) y suelen tener confianzas de entre 94 % y 100 %. Las predicciones con menor confianza corresponden precisamente a los casos ambiguos identificados en la matriz de confusión: un glaciar rodeado de montañas (59 %) y una calle antigua en blanco y negro flanqueada por edificios, clasificada como `buildings` con 61 %. Se observa además que el modelo asigna probabilidades del 100 % con frecuencia, lo que es consistente con la sobreconfianza detectada en la curva de pérdida de validación.

6. Diagnóstico y Mejoras Propuestas

El análisis de resultados permite identificar tres problemas principales del modelo base, cada uno asociado a una técnica de mejora concreta:

Cuadro 8: Problemas identificados y técnicas de mejora propuestas.

Problema	**Evidencia**	**Técnica propuesta**
Sobreajuste	Brecha de 19 puntos entre train y validación; *val loss* se triplica desde la época 5.	*Data augmentation* (volteo horizontal, rotación, zoom, contraste), *dropout* y *early stopping*.
Exceso de parámetros en el clasificador	84.6 % de los parámetros en una sola capa densa.	Reemplazar `Flatten` por `GlobalAveragePooling2D` y añadir *batch normalization*.
Características insuficientes para clases similares	50 % de los errores en los pares glaciar–montaña y edificios–calle.	*Transfer learning* con una red preentrenada en ImageNet (p. ej. EfficientNetB0).

- **Data augmentation.** Genera variaciones realistas de cada imagen de entrenamiento en cada época, de modo que el modelo no puede memorizar ejemplos específicos y debe aprender características invariantes. Se descarta el volteo vertical, ya que un paisaje invertido no es una imagen realista.
- **Dropout.** Desactiva aleatoriamente una fracción de las neuronas durante el entrenamiento, evitando que la red dependa de combinaciones específicas de neuronas.
- **Early stopping.** Detiene el entrenamiento cuando la pérdida de validación deja de mejorar, evitando las épocas en las que el modelo solo memoriza.
- **Global Average Pooling.** Promedia cada mapa de activación en lugar de aplanarlo, lo que reduce la capa densa de 1.3 millones de parámetros a unos pocos miles.
- **Transfer learning.** Aprovecha filtros ya entrenados con más de un millón de imágenes, capaces de distinguir texturas y formas finas (como hielo frente a roca) que un modelo entrenado con 11,000 imágenes difícilmente aprende por sí solo.

Cada técnica se aplicará de forma incremental y por separado, comparando siempre contra el modelo base sobre el mismo conjunto de validación, con el fin de documentar el efecto individual de cada cambio.

7. Conclusiones

Se implementó una red neuronal convolucional de cuatro bloques convolucionales y 1.57 millones de parámetros para clasificar escenas naturales en seis categorías. El modelo base alcanzó una exactitud de 83.33 % y un F1-score macro de 0.8355 en el conjunto de prueba, muy por encima del 17.50 % de un clasificador trivial, con un desempeño consistente entre validación y prueba.

El análisis de resultados reveló dos limitaciones claras. La primera es un sobreajuste marcado a partir de la sexta época, causado por la ausencia de regularización y por la concentración de parámetros en la capa densa. La segunda es la dificultad para separar clases visualmente similares, ya que los pares glaciar–montaña y edificios–calle concentran la mitad de los errores. Ambas limitaciones orientan directamente la siguiente etapa del proyecto: aplicar técnicas de regularización para cerrar la brecha entre entrenamiento y validación, y utilizar *transfer learning* para mejorar la discriminación entre las clases más difíciles.